A Mini Project Report on

E-Voting dApp on the Ethereum Blockchain

Submitted By

Rohit Sharma

Class: BE Computer B

Subject: Blockchain Technology (Mini Project)

Submitted to: Dr. Sagar Rane

Submitted as a partial fulfillment towards the subject of

B.E. Computer Engineering



ARMY INSTITUTE OF TECHNOLOGY

Department of Computer Engineering



Savitribai Phule Pune University

2026-27

ARMY INSTITUTE OF TECHNOLOGY

Department of Computer Engineering



C E R T I F I C A T E

This is to certify that **Rohit Sharma** of **BE Computer B**, Army Institute of Technology, Dighi, Pune has successfully completed the mini project entitled "**E-Voting dApp on the Ethereum Blockchain**" as a partial fulfillment of the subject **Blockchain Technology (Mini Project)** for the award of the degree of **Bachelor of Engineering (Computer Engineering)**, Savitribai Phule Pune University, Pune, during the Academic Year **2026-27**.

Dr. Sagar Rane
Subject Teacher

Prof. (Dr.) Sunil Dhore
H.O.D.

Acknowledgement

I express my sincere gratitude to my subject teacher **Dr. Sagar Rane** for the guidance and encouragement given throughout this mini project. I also thank the Head of the Department, **Prof. (Dr.) Sunil Dhore**, and the faculty of the **Department of Computer Engineering**, Army Institute of Technology, Pune, for providing the facilities needed to complete the work. I am grateful to the open-source communities behind Ethereum, Solidity, Remix IDE, MetaMask and ethers.js, whose free tools and documentation made it possible to build and test this application. Finally, I thank my family and friends for their constant support.

Abstract

Traditional voting systems, whether paper based or run on a central server, ask voters to trust the organisation that counts the votes. This project presents a decentralised e-voting application (dApp) that removes that single point of trust. The election rules are written as a Solidity smart contract and deployed on the Ethereum Sepolia test network. An administrator adds candidates, registers eligible voter wallets and opens and closes the election. Each registered wallet can cast exactly one vote, and every vote and the final result are stored on the blockchain where anyone can verify them. A browser front end built with HTML, JavaScript and ethers.js connects to the contract through the MetaMask wallet and shows the ballot, live vote counts and the result.

The system was first tested on the Remix VM and then on Sepolia, including negative tests such as a repeated vote and an unauthorised admin call, both of which the contract rejected. In the final Sepolia run two votes were recorded through the web page and the winner, FlankerH, was published with 2 votes. The report also records the transaction fees observed in MetaMask and discusses limitations such as public votes and the trust placed in the administrator.

Keywords: blockchain, Ethereum, smart contract, Solidity, dApp, e-voting, MetaMask, Remix, Sepolia, ethers.js

Table of Contents

1. Introduction	4
2. Theoretical Background	4
3. System Analysis	6
4. System Design	6
5. Implementation	8
6. Testing, Results and Discussion	9
7. Limitations and Future Scope	15
8. Conclusion	16
References	16
Appendix A: Smart Contract Code (Voting.sol)	17
Appendix B: Front-end Code (index.html)	19

List of Figures

Fig 1: Architecture of the e-voting dApp	7
Fig 2: Phases of the election and the functions allowed in each	7
Fig 3: Flow of the vote validation inside the contract	8
Fig 4: Admin (Account 1) adds candidates; three addCandidate transactions succeed (green) and each emits one log	9
Fig 5: registerVoters succeeds with logs: 2 (two VoterRegistered events); getCandidates is called in between	10
Fig 6: Full transaction list: constructor, three candidates, registerVoters and startVoting all succeed	10
Fig 7: Account 2 tries to vote again: the transaction is rejected with the reason "You have already voted"	10
Fig 8: getCandidates after Account 2's vote: Flanker has 1 vote, Fishbed 0, Foxbat 0	10
Fig 9: vote(5) with an invalid id from Account 2 is rejected; the already-voted check runs first, so that message is shown	10
Fig 10: Admin ends voting; rejected calls show "Only admin can do this"; phase returns 2 (Ended)	10
Fig 11: getWinner after the election ended returns Flanker with 1 vote	10
Fig 12: MetaMask Transaction request for addCandidate on Sepolia, interacting with the contract 0x46570...889d7	11
Fig 13: Console showing a failed transaction (red) followed by a successful retry (green); failed attempts still use gas	11
Fig 14: addCandidate and registerVoters confirmed on Sepolia; MetaMask shows Transaction confirmed	11
Fig 15: getCandidates returns FlankerH (id 0) and Fishbed (id 1) with 0 votes	12
Fig 16: candidateCount returns 2	12
Fig 17: admin returns the deployer wallet 0x28459d...01Be4	12
Fig 18: hasVoted of a voter wallet returns false	12
Fig 19: isRegistered of the same wallet returns true	12
Fig 20: phase returns 0 (Registration)	12
Fig 21: totalVotes returns 0	12
Fig 22: startVoting confirmed; phase now returns 1 (Voting)	12
Fig 23: MetaMask asks to connect the page (127.0.0.1) to Account 1	13
Fig 24: Review permissions: use the Sepolia network	13
Fig 25: Contract loaded: role Admin, phase Voting, 0 votes cast, candidates FlankerH and Fishbed with Vote buttons and the Admin panel	13
Fig 26: Clicking Vote opens a MetaMask Transaction request; the message bar asks the user to confirm	13
Fig 27: Vote confirmed: transaction hash shown, FlankerH has 1 vote (100%), the VOTE RECORDED stamp appears and the Vote buttons are disabled	13
Fig 28: A second vote has been recorded (2 votes cast); the admin now clicks End voting and MetaMask asks for confirmation	14
Fig 29: Election ended: the Result box announces FlankerH with 2 votes and the admin buttons are disabled	14
Fig 30: phase returns 2 (Ended)	14
Fig 31: getWinner returns name FlankerH and votes 2	14

1. Introduction

1.1 Background

Voting is the foundation of democratic decision making, whether in a national election, a college council or a society meeting. Paper ballots are trusted because people can watch the process, but counting is slow, printing and staffing are costly, and results can be disputed. Online voting removes many of these costs, yet most online systems store votes in a database controlled by one organisation. Voters must then trust that the administrator, and anyone who can break into the server, will not change or delete records.

Blockchain technology offers a different model. A blockchain is a shared ledger copied across thousands of independent computers. Records are added in blocks that are linked by cryptographic hashes, so an earlier record cannot be altered without every later block changing too. When the rules of an election are placed inside a smart contract on such a ledger, the counting logic is public, runs the same way for everybody and cannot be quietly edited.

1.2 Problem Statement

Centralised e-voting systems have a single point of failure and a single point of trust. Votes and tallies can be altered by insiders or attackers, voters cannot independently verify the result, and the system stops when the server fails. The problem addressed in this project is to design and build a voting application in which eligibility rules, vote recording and result calculation are enforced by a smart contract on a public blockchain, and in which every step can be inspected by anyone.

1.3 Objectives

- Design an election with three phases (Registration, Voting, Ended) that only the administrator can change.
- Allow only registered wallet addresses to vote, and allow each of them to vote once.
- Store candidates, vote counts and the winner on the blockchain so that results can be verified.
- Build a browser based dApp that talks to the smart contract through the MetaMask wallet.
- Test the contract on Remix VM and on the Sepolia test network, including failure cases, and record the fee behaviour.

1.4 Scope

The project covers the smart contract, the web front end, deployment on Sepolia and testing with real MetaMask accounts. It does not verify the real-world identity of voters, does not hide how a wallet voted (votes on a public blockchain are visible) and runs only on a test network, so no real money is involved. These limits are discussed in Chapter 7.

1.5 Organisation of the Report

Chapter 2 explains the technologies used. Chapter 3 analyses the problem and lists the requirements. Chapter 4 presents the design with diagrams and tables. Chapter 5 describes the implementation of the contract and the web page. Chapter 6 shows the test results with screenshots. Chapter 7 discusses limitations and future work, and Chapter 8 concludes. The complete source code is given in Appendices A and B.

2. Theoretical Background

2.1 Blockchain

A blockchain is an append-only database maintained by many computers (nodes) that do not need to trust each other. Transactions are grouped into blocks. Each block stores the hash of the previous block, so changing an old record would change its hash and break every block after it. Nodes follow a consensus protocol to agree on which block is added next. The main properties used in this project are **immutability** (recorded data is practically impossible to change), **transparency** (all nodes can read the same data) and **decentralisation** (no single owner of the database).

2.2 Ethereum

Ethereum is a public blockchain platform that can run programs as well as record payments. It has two kinds of accounts: externally owned accounts, controlled by a private key (such as a MetaMask account), and contract accounts, controlled by code. Programs run on the Ethereum Virtual Machine (EVM), which every node executes in the same way. Since 2022 Ethereum uses proof of stake, in which validators lock up ether as a guarantee of honest behaviour, and a new block appears about every 12 seconds. The currency is ether (ETH); its smallest unit is the wei (1 ETH = 10^18 wei).

2.3 Smart Contracts

A smart contract is a program stored at an address on the blockchain. Once deployed its code cannot be changed, and it runs exactly as written whenever someone sends it a transaction. Contract functions can read and update the contract's own storage, check who the caller is (*msg.sender*), reject a call with *require* and publish *events*, which are searchable log entries. Because the contract holds the election rules, nobody, including the administrator, can bypass them.

2.4 Solidity

Solidity is the main language for writing Ethereum smart contracts. It is statically typed and resembles JavaScript and C++. The features used in this project are: **enum** for the election phase, **struct** for a candidate, **mapping** for the registered and voted flags, **arrays** for the candidate list, **modifiers** (onlyAdmin, inPhase) for access control, **constructor** to set the administrator, **events** for logging, and visibility keywords such as public, external and private. Functions marked *view* only read data.

2.5 Gas and Transaction Fees

Every operation in the EVM costs gas, which prevents infinite loops and pays the validators. The fee for a transaction is **gas used x gas price**, paid in ETH by the sender. Writing to storage is the most expensive operation, so deploying a contract costs the most, and a plain ETH transfer costs the minimum of 21,000 gas. Reading data with a call does not create a transaction and is free. A transaction that fails (reverts) still consumes gas for the work done before the failure.

2.6 Wallets and MetaMask

A wallet manages the private key of an account and signs transactions. MetaMask is a browser extension wallet. It shows every transaction request with the network, the contract being used and the estimated fee, and it signs only after the user presses Confirm. Web pages never see the private key; they can only ask MetaMask to sign. MetaMask can also hold several accounts and switch between networks, which is how one person can act as the administrator and as different voters during testing.

2.7 Test Networks and Faucets

Sepolia is an Ethereum test network (chain ID 11155111) that works like the main network but uses test ETH with no real value. Developers receive test ETH free from a faucet website and use it to pay gas. Dollar amounts shown by MetaMask on a test network are only conversions of the test token and do not represent real cost.

2.8 Remix IDE

Remix is a browser based development environment for Solidity. It compiles contracts, deploys them and gives a button for every function so that they can be tested without writing a front end. Two environments were used: **Remix VM**, a simulated chain inside the browser with funded test accounts, and **Browser Extension**, which sends transactions through MetaMask to a real network such as Sepolia.

2.9 ethers.js and dApp Architecture

ethers.js is a JavaScript library for talking to Ethereum. The front end creates a *BrowserProvider* from the wallet injected by MetaMask, obtains a *signer* for the connected account and builds a *Contract* object from the contract address and its ABI (the list of functions). Calling a view function returns data; calling a state-changing function asks MetaMask to sign and send a transaction. A decentralised application therefore has no application server: the web page is only a user interface and the contract on the blockchain is the back end.

3. System Analysis

3.1 Existing System and its Limitations

In a conventional online election one organisation runs a web server and a database. The administrator registers voters, the server accepts votes and the same organisation announces the result. The limitations are: votes can be changed by anyone with database access; voters cannot check that their vote was counted; the system can be switched off or attacked; and trust rests on one party.

3.2 Proposed System

The proposed system moves the election rules and the vote tally into a smart contract. The administrator is still needed to prepare the election, but the administrator cannot change a recorded vote, cannot let a wallet vote twice and cannot hide the result. Anyone can read candidates, counts and the phase directly from the blockchain.

Criterion	Conventional online voting	Proposed blockchain dApp
Who counts votes	Central server run by the organiser	Smart contract code, run by all nodes
Tamper resistance	Depends on database security	Records replicated and hash linked
Transparency	Voters must trust the audit	Anyone can read votes and totals
Availability	One server can fail or be attacked	Network of independent nodes
Voter eligibility	Identity checks by the organiser	Wallet addresses registered by the admin
Ballot secrecy	Can be hidden by the system	Votes are public (a limitation)
Cost	Servers and staff	Gas fees paid in ETH

3.3 Requirements

ID	Functional requirement
FR1	The admin can add candidates during the Registration phase.
FR2	The admin can register voter wallet addresses, one or many at a time.
FR3	The admin can start voting only when at least two candidates exist.
FR4	A registered voter can vote for one candidate while voting is open.
FR5	No wallet can vote twice, and an unregistered wallet cannot vote.
FR6	The admin can end the election; after that no votes are accepted.
FR7	Anyone can read the candidates, vote counts, phase and winner.
FR8	The web page shows the user's role, the phase, the ballot and the result.

Non-functional requirements: integrity of recorded votes, transparency of results, access control for admin actions, availability without a central server, a simple user interface, and low gas use (one storage write per vote).

3.4 Actors

Actor	Who	Can do
Admin	The wallet that deploys the contract	Add candidates, register voters, start and end voting
Voter	A wallet registered by the admin	Cast one vote while voting is open, view results
Observer	Anyone	Read candidates, counts, phase and winner

4. System Design

4.1 Architecture

The application has three layers, shown in Fig 1. The **web page** is the user interface. **MetaMask** holds the keys, asks the user to confirm and signs transactions. The **Voting contract** lives on the Sepolia network and stores the election. Remix is used to compile and deploy the contract, a faucet supplies test ETH for gas, and Etherscan lets anyone look at the transactions.

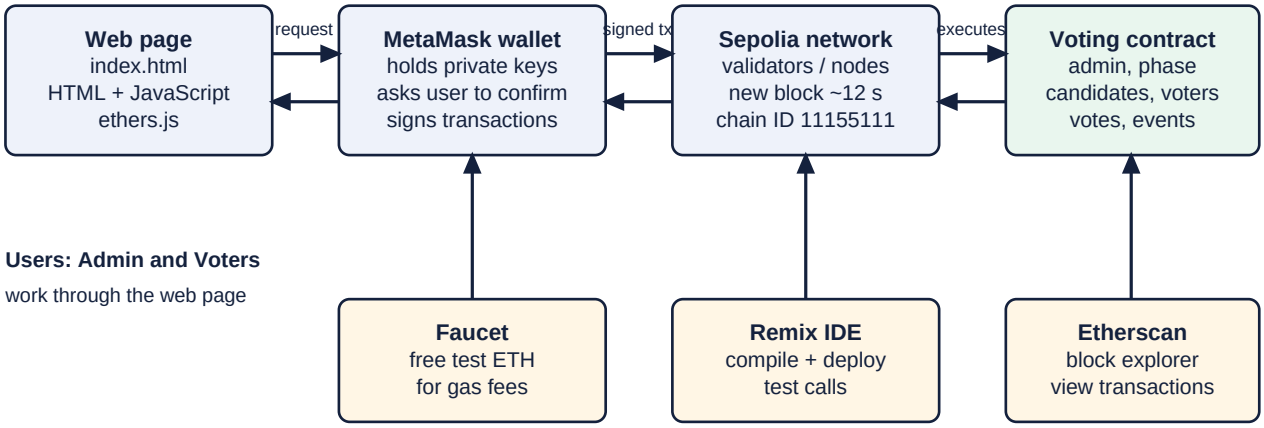


Fig 1: Architecture of the e-voting dApp

4.2 Election Phases

The election moves through three phases (Fig 2). Setup functions only work in Registration, *vote* only works in Voting, and the winner can be read only after the election has Ended. The phase can only move forward, and only the admin can move it. This prevents, for example, adding a candidate after voting has started.

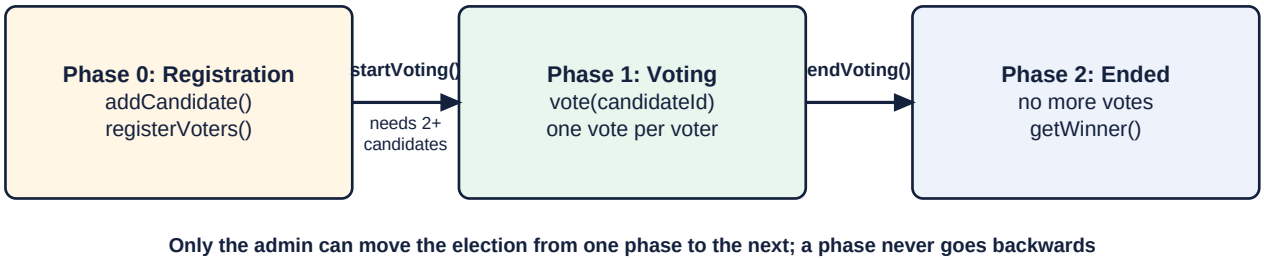


Fig 2: Phases of the election and the functions allowed in each

4.3 Data Design

Variable	Type	Purpose
admin	address	Wallet that deployed the contract and manages the election
phase	enum Phase	Registration (0), Voting (1) or Ended (2)
totalVotes	uint256	Number of votes cast so far
candidates	Candidate[]	List of candidates; each has id, name and voteCount
isRegistered	mapping(address => bool)	True if the wallet is allowed to vote
hasVoted	mapping(address => bool)	True once the wallet has voted

4.4 Vote Validation

When a voter calls *vote*, the contract runs the checks in Fig 3 in order. Any failed check reverts the transaction with a readable message, so no state changes. Only when all checks pass is the vote recorded and the *Voted* event emitted.

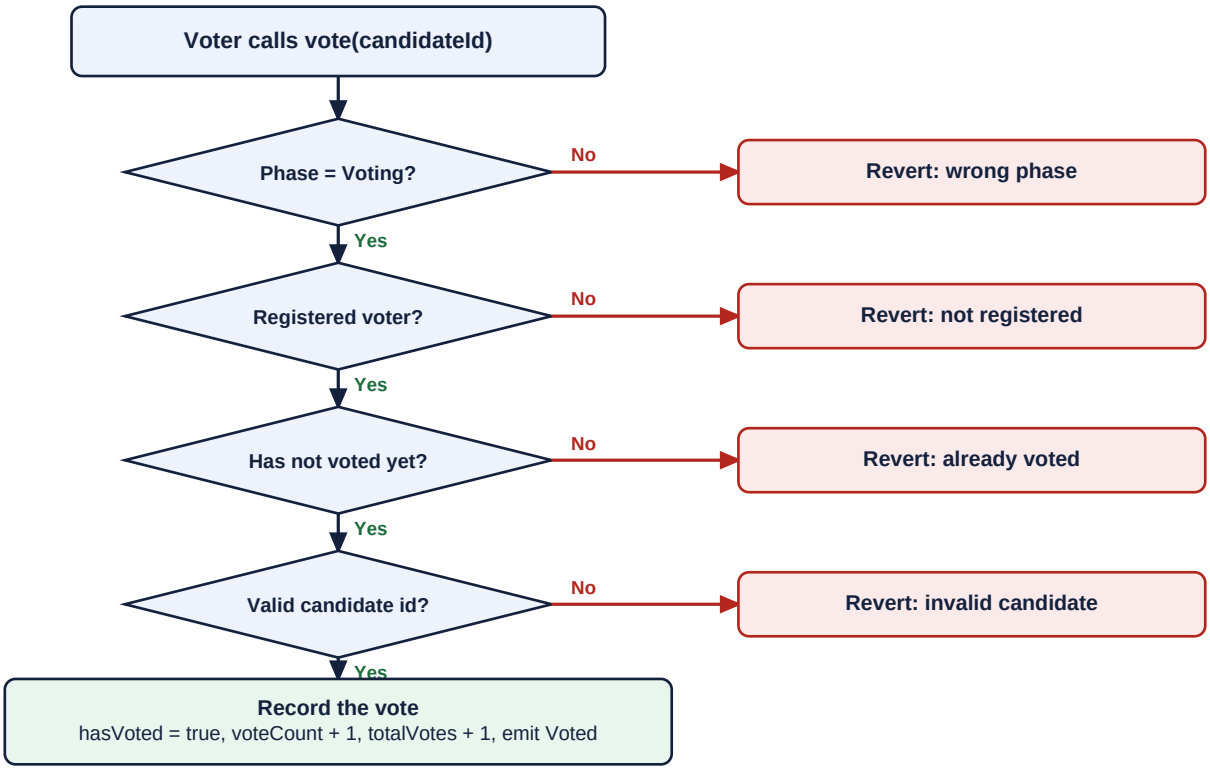


Fig 3: Flow of the vote validation inside the contract

4.5 Functions and Events

Function	Caller	Phase	Purpose
addCandidate(name)	Admin	Registration	Adds a candidate with id and zero votes
registerVoters(list)	Admin	Registration	Marks the given wallets as eligible
startVoting()	Admin	Registration	Opens voting; needs at least 2 candidates
vote(candidateId)	Registered voter	Voting	Records one vote
endVoting()	Admin	Voting	Closes the election
getCandidates()	Anyone	Any	Returns all candidates with their counts
candidateCount()	Anyone	Any	Number of candidates
getWinner()	Anyone	Ended	Winner's name and votes (first candidate wins a tie)

Public getters are also generated for *admin*, *phase*, *totalVotes*, *isRegistered* and *hasVoted*. The contract emits four events: CandidateAdded, VoterRegistered, PhaseChanged and Voted.

5. Implementation

5.1 Tools and Environment

Item	Details
Smart contract language	Solidity ^0.8.20, file Voting.sol
Development environment	Remix IDE 2.6.6 in the browser (Brave)
Test environments	Remix VM (Osaka) and Sepolia via the Browser Extension option
Wallet	MetaMask browser extension, several accounts
Front end	HTML, CSS and JavaScript with ethers.js 6.13.4 loaded from a CDN
Local web server	Live server on 127.0.0.1:5500 (as seen in the screenshots)
Network	Ethereum Sepolia test network, chain ID 11155111
Sepolia contract address	0x46570EeB397B84A6371Af12B1820Ede59a9889d7
Admin wallet	0x28459d2F5fa871fDe0c389B88b88F38d9E001Be4

5.2 Smart Contract

The complete contract is listed in Appendix A. It begins with the **Phase** enum and the **Candidate** struct. State variables hold the admin, the phase, the total vote count, the candidate array and the two mappings. The constructor sets *admin = msg.sender* and the phase to Registration, which is why the wallet used for deployment becomes the administrator.

Two modifiers implement access control. *onlyAdmin* reverts with "Only admin can do this" when the caller is not the admin, and *inPhase(p)* reverts with "Not allowed in the current phase". The setup functions *addCandidate* and *registerVoters* use both modifiers. *startVoting* additionally requires at least two candidates, and *endVoting* is allowed only while voting is open.

The *vote* function is the core of the system. It requires the Voting phase, then checks that the caller is registered, that the caller has not voted and that the candidate id exists. It then sets *hasVoted[msg.sender] = true* before increasing the candidate's count and the total, and emits the *Voted* event. Setting the flag first is a good habit that prevents re-entrancy style problems. Because the checks run in this order, a wallet that has already voted gets the message "You have already voted" even if it also supplies an invalid candidate id.

5.3 Web Front End

The complete page is listed in Appendix B. The *connect()* function asks MetaMask for the account, switches the wallet to Sepolia if needed and creates the provider and signer. *loadContract()* builds the contract object from the address typed by the user. *refresh()* reads the admin, phase, total votes, the user's registration and voting status and the candidate list, then redraws the page. The Admin panel is shown only when the connected wallet equals the contract's admin, and the Vote buttons are enabled only during the Voting phase for a registered wallet that has not yet voted.

State-changing actions go through one helper, *send()*. It asks the user to confirm in MetaMask, waits for the transaction to be mined, shows the transaction hash and refreshes the page. Error messages from the contract are shown to the user. Candidate names are written to the page with *textContent* so that a malicious name cannot inject HTML. The page reloads when the account or network changes and polls the contract every 12 seconds.

5.4 Deployment and Execution Procedure

1. Open Remix, create Voting.sol and compile it with a Solidity version of 0.8.20 or higher.
2. Test the contract on Remix VM using its funded test accounts (Section 6.2).
3. Switch the environment to Browser Extension, choose Sepolia Testnet - MetaMask and deploy from the admin account.
4. Copy the deployed contract address. As admin, add candidates, register voter wallets and start voting.
5. Serve index.html from a local web server, open it in the browser that has MetaMask and connect the wallet.
6. Paste the contract address, load the election, vote with registered accounts and finally end the voting as admin.

6. Testing, Results and Discussion

6.1 Test Strategy

Testing was done in two stages. In Stage 1 the contract was deployed on **Remix VM** and exercised function by function with several test accounts, including failure cases. In Stage 2 the same logic was deployed on the **Sepolia** test network and used with real MetaMask accounts, first through Remix and then through the web page. Screenshots in this chapter have been converted to a white background to save ink.

6.2 Stage 1: Tests on Remix VM

The contract was deployed from Account 1 (the admin) on Remix VM. Three candidates (Flanker, Fishbed and Foxbat) were added, two voters were registered and voting was started. Account 2 then voted and the later tests tried to break the rules.

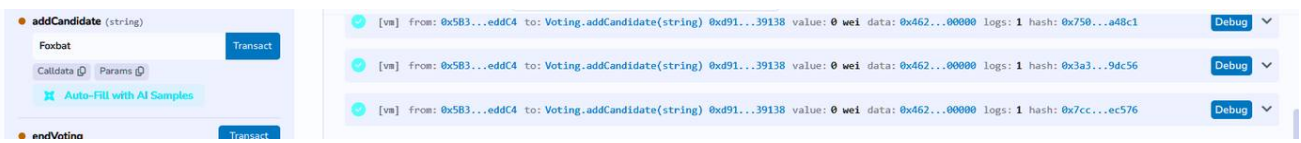


Fig 4: Admin (Account 1) adds candidates; three addCandidate transactions succeed (green) and each emits one log

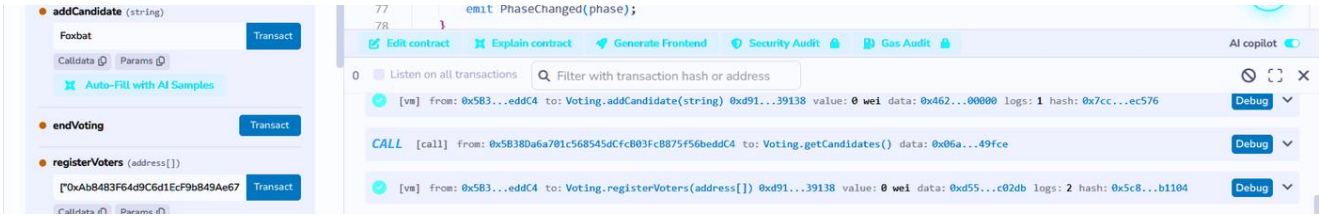


Fig 5: registerVoters succeeds with logs: 2 (two VoterRegistered events); getCandidates is called in between



Fig 6: Full transaction list: constructor, three candidates, registerVoters and startVoting all succeed

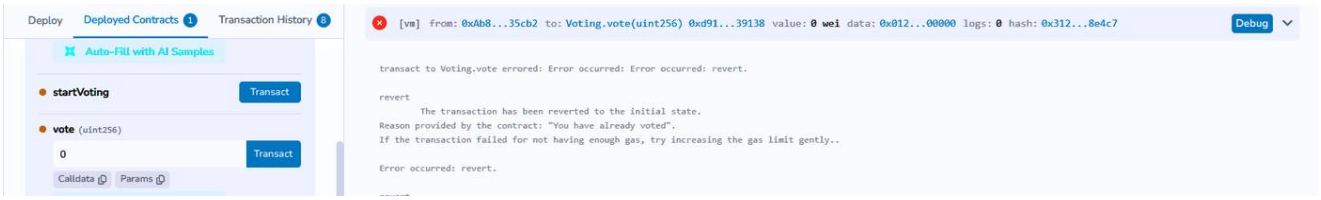


Fig 7: Account 2 tries to vote again: the transaction is rejected with the reason "You have already voted"

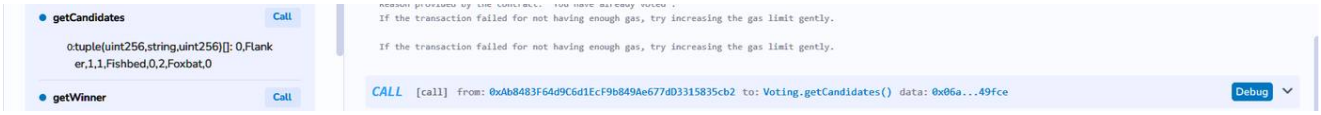


Fig 8: getCandidates after Account 2's vote: Flanker has 1 vote, Fishbed 0, Foxbat 0

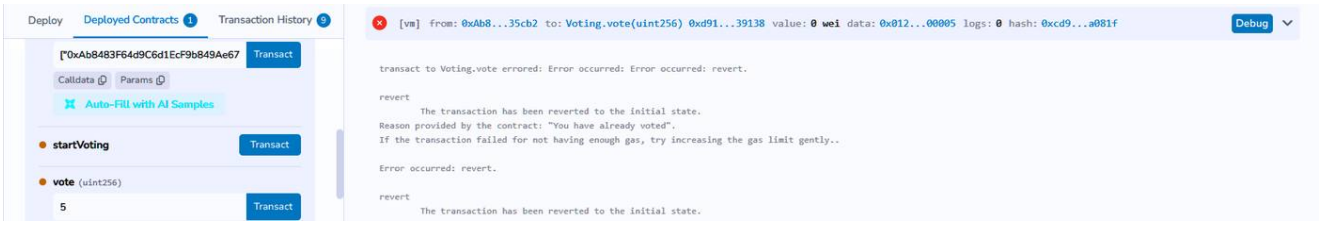


Fig 9: vote(5) with an invalid id from Account 2 is rejected; the already-voted check runs first, so that message is shown

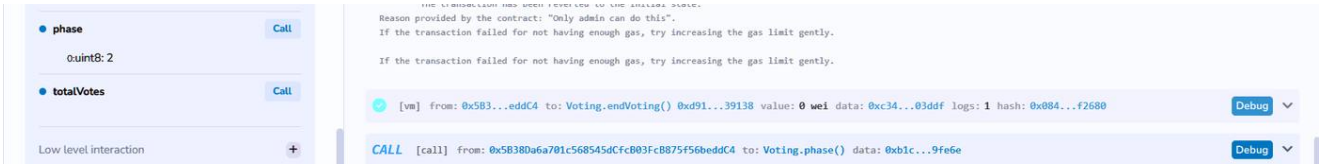


Fig 10: Admin ends voting; rejected calls show "Only admin can do this"; phase returns 2 (Ended)

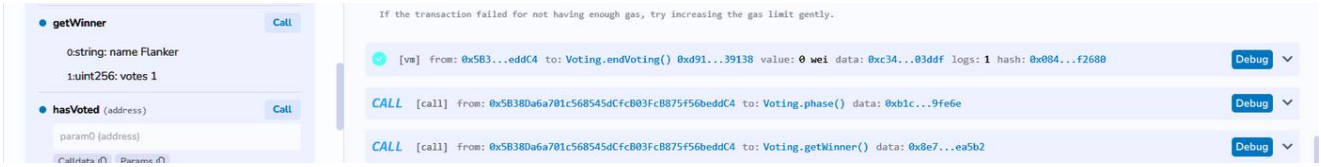


Fig 11: getWinner after the election ended returns Flanker with 1 vote

6.3 Stage 2: Deployment on Sepolia with MetaMask

The contract was deployed on Sepolia from the admin MetaMask account. Every state-changing call opens a MetaMask Transaction request that shows the network, the contract address and the estimated fee.

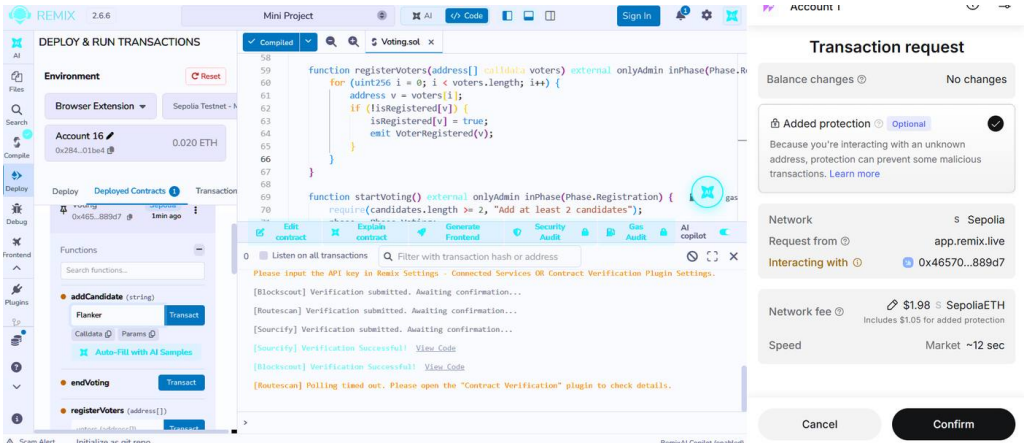


Fig 12: MetaMask Transaction request for addCandidate on Sepolia, interacting with the contract 0x46570...889d7

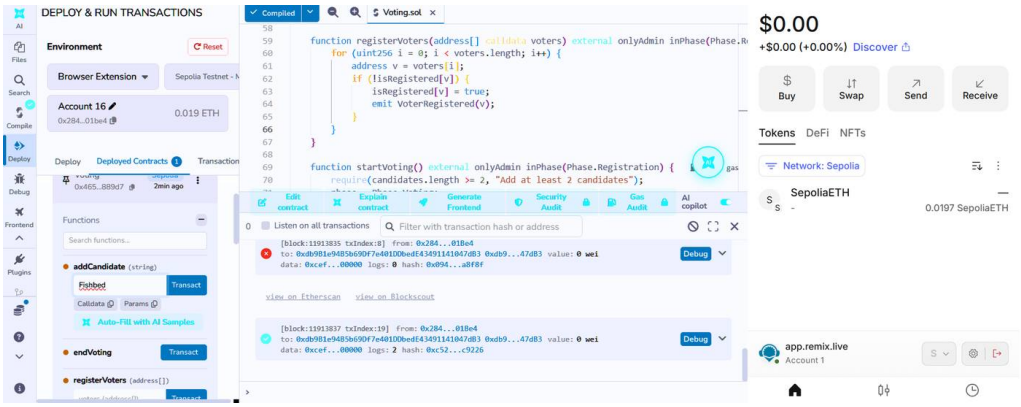


Fig 13: Console showing a failed transaction (red) followed by a successful retry (green); failed attempts still use gas

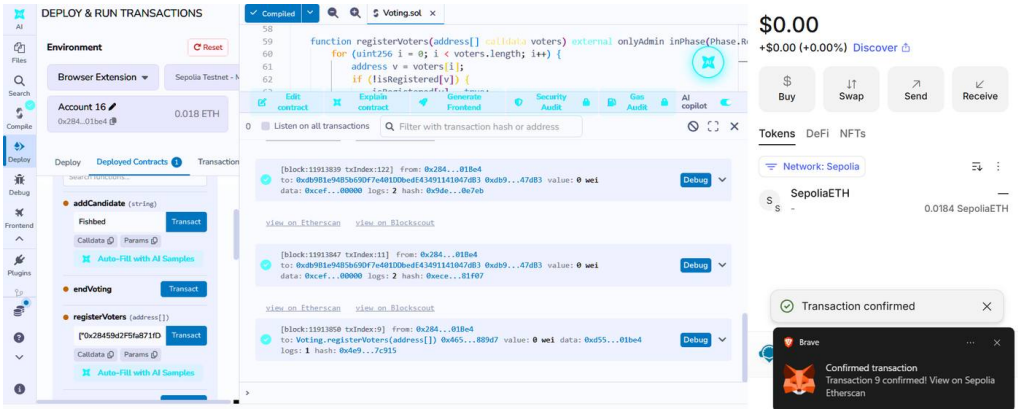


Fig 14: addCandidate and registerVoters confirmed on Sepolia; MetaMask shows Transaction confirmed

After the setup transactions the public read functions were called. They are free and need no MetaMask confirmation.

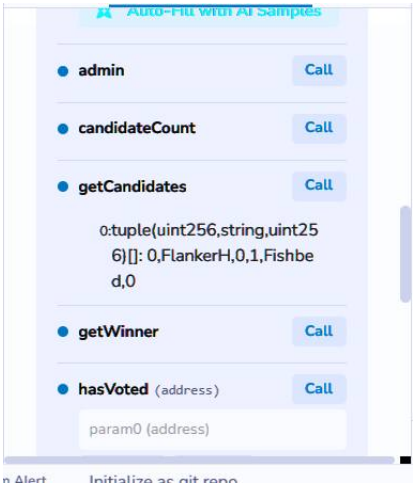


Fig 15: getCandidates returns FlankerH (id 0) and Fishbed (id 1) with 0 votes

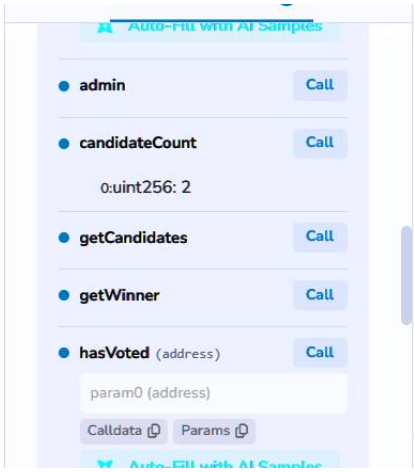


Fig 16: candidateCount returns 2

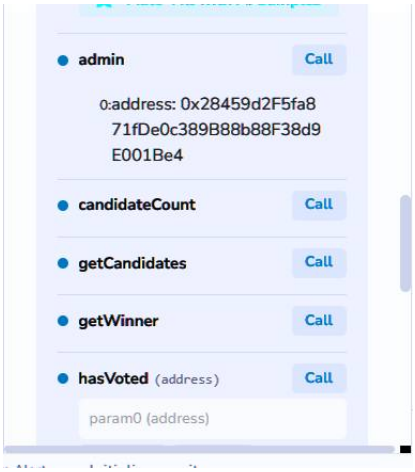


Fig 17: admin returns the deployer wallet 0x28459d...01Be4

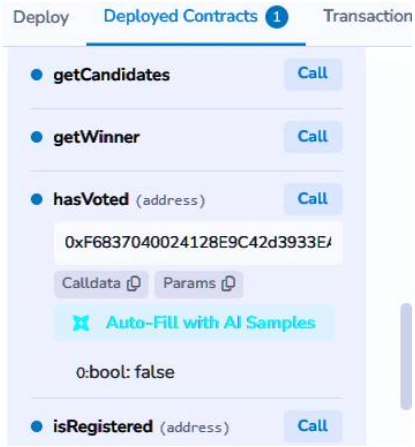


Fig 18: hasVoted of a voter wallet returns false

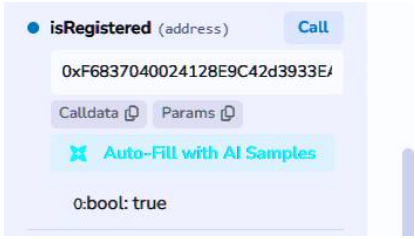


Fig 19: isRegistered of the same wallet returns true



Fig 20: phase returns 0 (Registration)

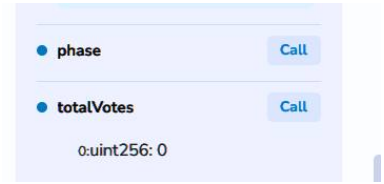


Fig 21: totalVotes returns 0



Fig 22: startVoting confirmed; phase now returns 1 (Voting)

6.4 Stage 2: Using the Web dApp

The page was served locally and opened in the browser that has MetaMask. The first step is to connect the wallet; the page requests permission and asks to use the Sepolia network.

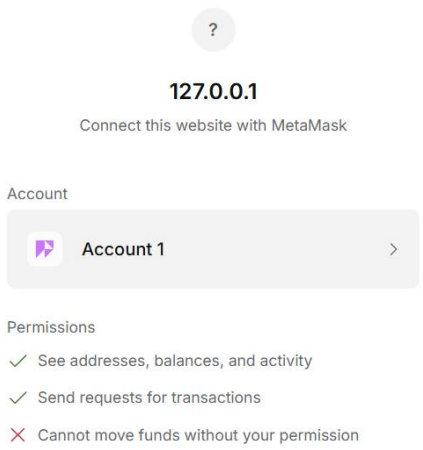


Fig 23: MetaMask asks to connect the page (127.0.0.1) to Account 1

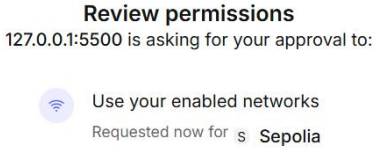


Fig 24: Review permissions: use the Sepolia network

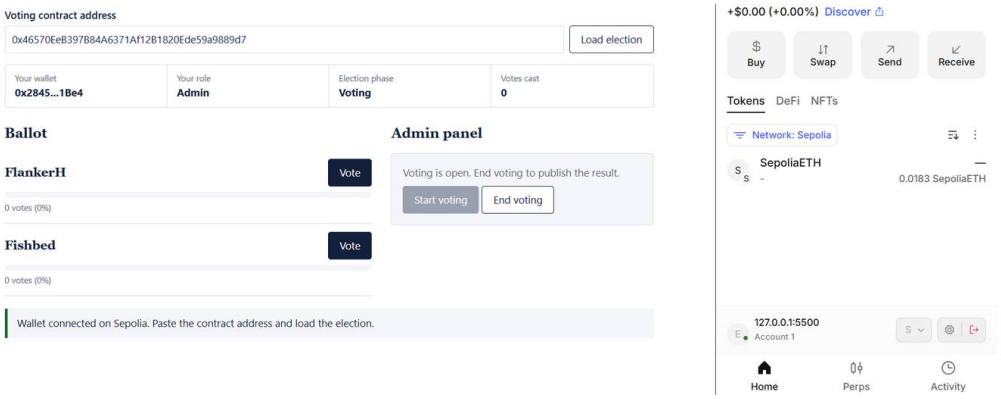


Fig 25: Contract loaded: role Admin, phase Voting, 0 votes cast, candidates FlankerH and Fishbed with Vote buttons and the Admin panel

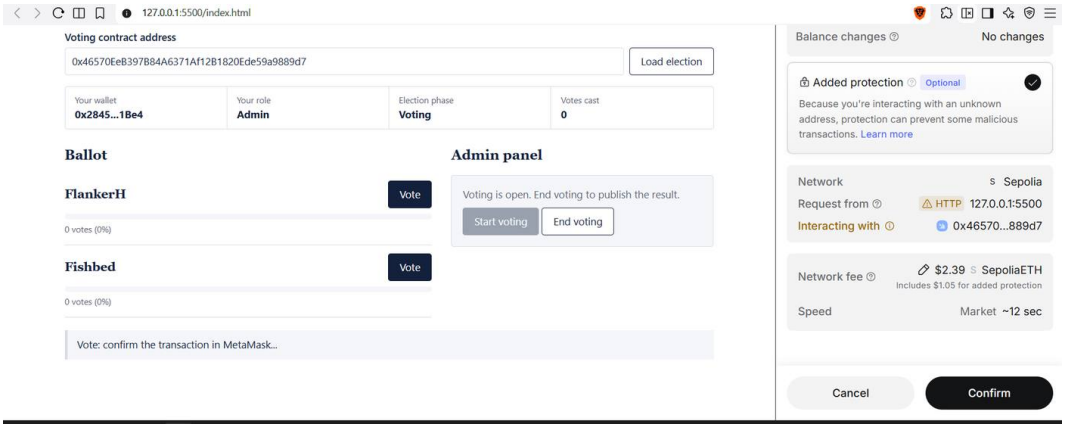


Fig 26: Clicking Vote opens a MetaMask Transaction request; the message bar asks the user to confirm

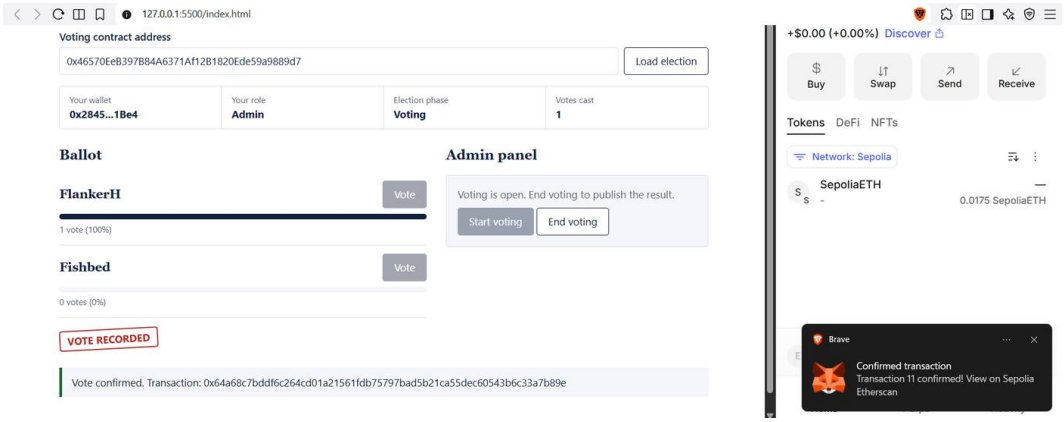


Fig 27: Vote confirmed: transaction hash shown, FlankerH has 1 vote (100%), the VOTE RECORDED stamp appears and the Vote buttons are disabled

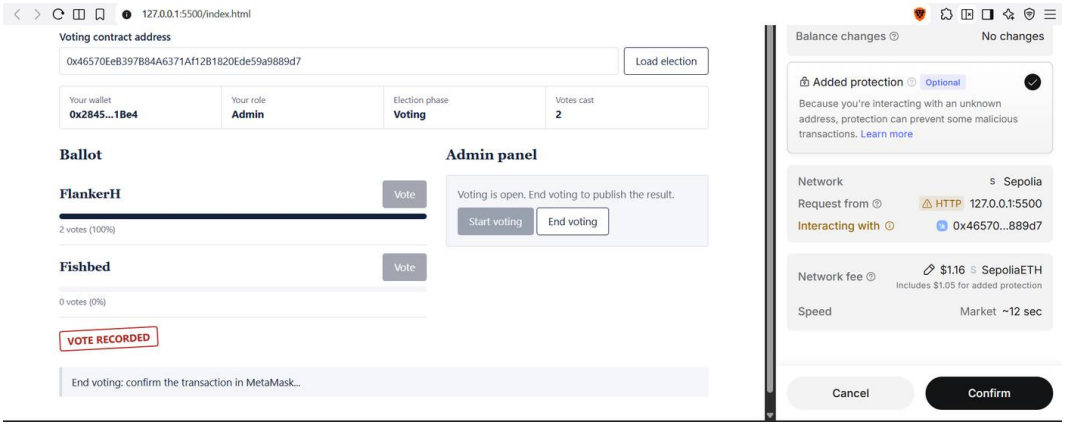


Fig 28: A second vote has been recorded (2 votes cast); the admin now clicks End voting and MetaMask asks for confirmation

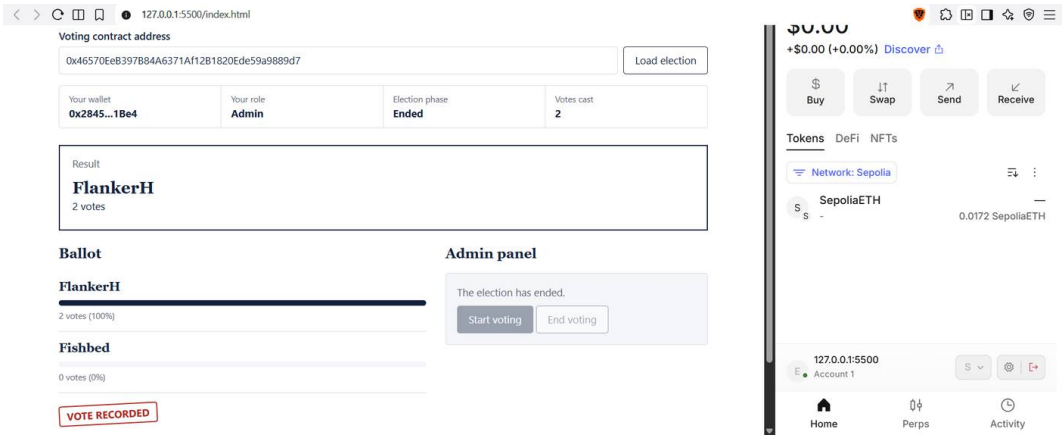


Fig 29: Election ended: the Result box announces FlankerH with 2 votes and the admin buttons are disabled

Finally the result was cross-checked directly on the contract from Remix, independent of the web page.

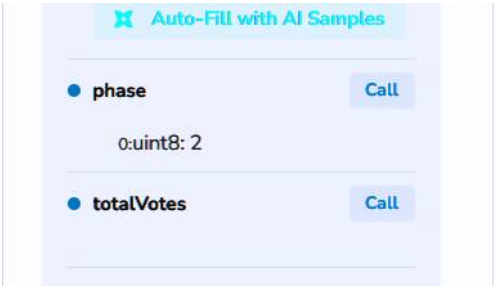


Fig 30: phase returns 2 (Ended)



Fig 31: getWinner returns name FlankerH and votes 2

6.5 Test Case Summary

No.	Test	Expected result	Observed (evidence)
1	Admin adds candidates	Candidates stored with 0 votes	Passed (Fig 4, Fig 15)
2	Admin registers voters	Wallets become registered	Passed (Fig 5, Fig 19)
3	Start voting with 2+ candidates	Phase becomes Voting	Passed (Fig 6, Fig 22)
4	Registered voter votes	Count of the chosen candidate increases	Passed (Fig 8, Fig 27)
5	Same wallet votes again	Rejected: You have already voted	Passed (Fig 7)
6	Invalid candidate id after voting	Transaction rejected	Passed, already-voted message shown (Fig 9)
7	Admin-only function without admin rights	Rejected: Only admin can do this	Passed (Fig 10)
8	Admin ends the election	Phase becomes Ended	Passed (Fig 10, Fig 30)
9	Read the winner	Winner and votes returned	Passed (Fig 11, Fig 31)
10	Free read calls on Sepolia	Correct data, no confirmation needed	Passed (Fig 15, Fig 16, Fig 17, Fig 18, Fig 20, Fig 21)
11	Web page connects and loads	Role, phase and ballot displayed	Passed (Fig 23, Fig 24, Fig 25)
12	Vote through the web page	Count increases, vote stamp, buttons disabled	Passed (Fig 26, Fig 27)
13	End voting through the web page	Result box shows the winner	Passed (Fig 28, Fig 29)

The contract also rejects an unregistered wallet, a vote outside the Voting phase and an invalid candidate id from a wallet that has not voted. These cases follow directly from the require statements in Appendix A but were not captured in the screenshots of this report.

6.6 Transaction Fee Observations

MetaMask displayed a fee estimate in dollars for each confirmation. These figures are conversions of the Sepolia test token and have no real value. Each estimate also included an optional added-protection item shown by MetaMask.

Action (MetaMask request)	Fee estimate shown	Of which optional protection	Figure
addCandidate	$1.98 (SepoliaETH)	$1.05	Fig 12
vote (web page)	$2.39 (SepoliaETH)	$1.05	Fig 26
endVoting (web page)	$1.16 (SepoliaETH)	$1.05	Fig 28

The wallet balance panels give a second view of the cost. The balance fell from 0.0197 SepoliaETH (Fig 13) to 0.0184 after the setup transactions (Fig 14) and 0.0183 when the page was loaded (Fig 25). After the vote it was 0.0175 (Fig 27) and after ending the election 0.0172 (Fig 29). So the vote cost roughly 0.0008 SepoliaETH and ending the election roughly 0.0003 SepoliaETH, although these are differences of balances rounded to four decimals and are only approximate.

6.7 Discussion

All functional requirements were met. The contract enforced the phase order, the admin-only functions and the one-vote rule in both stages, and the web page correctly reflected the on-chain state after each transaction. The Remix VM stage made it easy to try failure cases at no cost, while the Sepolia stage showed the real workflow: every action needs a MetaMask confirmation, takes about 12 seconds to be confirmed and costs a small fee. The failed transaction in Fig 13 is a useful reminder that a rejected transaction can still consume gas, which matters for a real election where voters must pay for their own votes.

7. Limitations and Future Scope

7.1 Limitations

- **No ballot secrecy:** transactions on a public blockchain show which wallet voted, and the Voted event records the chosen candidate id.

- **Admin trust:** the admin decides which wallets are eligible. A dishonest admin could register fake wallets, so identity checking is outside the contract.
- **Gas cost for voters:** every voter needs test ETH (or real ETH on a main network) to pay for the vote transaction.
- **No deadline:** the admin ends voting manually; there is no automatic closing time.
- **Ties:** if candidates tie, getWinner returns the first of them rather than reporting a tie.
- **Front-end dependency:** the page loads ethers.js from a CDN and runs on a local server; a production system would host and audit the page properly.
- **Test network only:** the contract has not been audited and should not be used for a real election as it is.

7.2 Future Scope

- Add a voting deadline using block timestamps so that the election closes automatically.
- Use zero-knowledge proofs or blind signatures to keep each vote secret while still proving that it was counted.
- Connect the registration step to a real identity system, such as a college ID check or a verifiable credential.
- Let voters pay no gas by using a relayer or a low-fee Layer 2 network.
- Support several elections in one contract, tie reporting and a history page built from the contract events.
- Host the front end on decentralised storage such as IPFS and get the contract audited.

8. Conclusion

This mini project built a complete decentralised e-voting application. The Solidity contract enforces a three-phase election, restricts management functions to the administrator, allows only registered wallets to vote and limits each wallet to one vote. A browser page using ethers.js and MetaMask lets the admin set up the election and lets voters cast votes and see the result. Testing on Remix VM and on the Sepolia network, with screenshots as evidence, confirmed correct behaviour including the rejection of a repeated vote and of unauthorised admin calls. The final run recorded two votes and announced FlankerH as the winner with 2 votes.

The project also gave practical experience with the full dApp workflow: writing and compiling a contract, deploying it with MetaMask, paying gas with test ETH, reading data for free, and building a user interface on top of the contract. The limitations listed above, mainly public votes and administrator trust, show where further work is needed before such a system could be used in a real election.

References

[1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", 2008.

[2] V. Buterin, "Ethereum White Paper: A Next-Generation Smart Contract and Decentralized Application Platform", 2014.

[3] G. Wood, "Ethereum: A Secure Decentralised Generalised Transaction Ledger" (Yellow Paper).

[4] Solidity Documentation, docs.soliditylang.org.

[5] ethers.js v6 Documentation, docs.ethers.org/v6.

[6] MetaMask Developer Documentation, docs.metamask.io.

[7] Remix IDE Documentation, remix-ide.readthedocs.io.

[8] Ethereum Developer Documentation, ethereum.org/developers.

Appendix A: Smart Contract Code (Voting.sol)

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

/**
 * @title Voting
 * @notice Simple e-voting system. The admin adds candidates and registers voters,
 *         then opens and closes the election. Each registered voter can vote once.
 * @dev For learning and test networks (Sepolia). Votes are public on-chain.
 */
contract Voting {
    // ---------- Types ----------
    enum Phase { Registration, Voting, Ended }

    struct Candidate {
        uint256 id;
        string name;
        uint256 voteCount;
    }

    // ---------- State ----------
    address public admin;
    Phase public phase;
    uint256 public totalVotes;

    Candidate[] private candidates;
    mapping(address => bool) public isRegistered;
    mapping(address => bool) public hasVoted;

    // ---------- Events ----------
    event CandidateAdded(uint256 indexed id, string name);
    event VoterRegistered(address indexed voter);
    event PhaseChanged(Phase newPhase);
    event Voted(address indexed voter, uint256 indexed candidateId);

    // ---------- Modifiers ----------
    modifier onlyAdmin() {
        require(msg.sender == admin, "Only admin can do this");
        _;
    }

    modifier inPhase(Phase p) {
        require(phase == p, "Not allowed in the current phase");
        _;
    }

    constructor() {
        admin = msg.sender;
        phase = Phase.Registration;
    }

    // ---------- Admin: set up the election ----------
    function addCandidate(string calldata name) external onlyAdmin inPhase(Phase.Registration) {
        require(bytes(name).length > 0, "Name is empty");
        uint256 id = candidates.length;
        candidates.push(Candidate(id, name, 0));
        emit CandidateAdded(id, name);
    }

    function registerVoters(address[] calldata voters) external onlyAdmin inPhase(Phase.Registration) {
        for (uint256 i = 0; i < voters.length; i++) {
            address v = voters[i];
            if (!isRegistered[v]) {
                isRegistered[v] = true;
                emit VoterRegistered(v);
            }
        }
    }

    function startVoting() external onlyAdmin inPhase(Phase.Registration) {
        require(candidates.length >= 2, "Add at least 2 candidates");
        phase = Phase.Voting;
        emit PhaseChanged(phase);
    }

    function endVoting() external onlyAdmin inPhase(Phase.Voting) {
        phase = Phase.Ended;
        emit PhaseChanged(phase);
    }

    // ---------- Voter ----------
    function vote(uint256 candidateId) external inPhase(Phase.Voting) {
        require(isRegistered[msg.sender], "You are not a registered voter");
        require(!hasVoted[msg.sender], "You have already voted");
        require(candidateId < candidates.length, "Invalid candidate");

        hasVoted[msg.sender] = true;
        candidates[candidateId].voteCount += 1;
        totalVotes += 1;

        emit Voted(msg.sender, candidateId);
    }

    // ---------- Read ----------
    function getCandidates() external view returns (Candidate[] memory) {
        return candidates;
    }
    function candidateCount() external view returns (uint256) {
```

```
        return candidates.length;
    }

    /// @notice Winner after the election ends (first candidate wins a tie).
    function getWinner() external view inPhase(Phase.Ended) returns (string memory name, uint256 votes) {
        require(candidates.length > 0, "No candidates");
        uint256 best = 0;
        for (uint256 i = 1; i < candidates.length; i++) {
            if (candidates[i].voteCount > candidates[best].voteCount) {
                best = i;
            }
        }
        return (candidates[best].name, candidates[best].voteCount);
    }
}
```

Appendix B: Front-end Code (index.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>E-Voting dApp</title>
<style>
  :root {
    --paper: #ffffff;
    --ink: #14213d;
    --muted: #5b6475;
    --rule: #d9dde5;
    --wash: #f4f6fa;
    --stamp: #b3261e;
    --ok: #1d6b3a;
    --serif: Georgia, "Times New Roman", serif;
    --sans: system-ui, -apple-system, "Segoe UI", Roboto, Arial, sans-serif;
  }
  * { box-sizing: border-box; }
  body { margin: 0; background: var(--paper); color: var(--ink); font: 15px/1.5 var(--sans); }
  main { max-width: 980px; margin: 0 auto; padding: 24px 20px 60px; }
  h1 { font: 700 30px/1.2 var(--serif); margin: 0; }
  h2 { font: 700 20px/1.3 var(--serif); margin: 0 0 12px; }
  p { margin: 0 0 8px; }
  .muted { color: var(--muted); }
  .top { display: flex; justify-content: space-between; align-items: flex-start; gap: 16px; flex-wrap: wrap;
         padding-bottom: 16px; border-bottom: 2px solid var(--ink); }
  button { font: inherit; cursor: pointer; border: 1.5px solid var(--ink); background: var(--ink); color: #fff;
           padding: 8px 16px; border-radius: 4px; }
  button.ghost { background: var(--paper); color: var(--ink); }
  button:disabled { opacity: .4; cursor: not-allowed; }
  button:focus-visible, input:focus-visible, textarea:focus-visible { outline: 3px solid #5b8def; outline-offset: 2px; }
  input, textarea { font: inherit; width: 100%; padding: 8px 10px; border: 1.5px solid var(--rule); border-radius: 4px; background:
#fff; color: var(--ink); }
  label { display: block; font-weight: 600; margin: 10px 0 4px; }
  .setup { display: grid; grid-template-columns: 1fr auto; gap: 8px; align-items: end; margin: 16px 0; }
  .status { display: grid; grid-template-columns: repeat(auto-fit, minmax(190px, 1fr)); gap: 0;
            border: 1px solid var(--rule); border-radius: 4px; margin-bottom: 24px; }
  .status div { padding: 10px 14px; border-right: 1px solid var(--rule); }
  .status div:last-child { border-right: 0; }
  .status small { display: block; color: var(--muted); }
  .status b { word-break: break-all; }
  .grid { display: grid; grid-template-columns: 1.4fr 1fr; gap: 28px; }
  @media (max-width: 760px) { .grid { grid-template-columns: 1fr; } .status div { border-right: 0; border-bottom: 1px solid var(--
rule); } }
  .candidate { padding: 12px 0; border-bottom: 1px solid var(--rule); }
  .candidate .row { display: flex; justify-content: space-between; align-items: center; gap: 12px; }
  .candidate .name { font: 700 18px/1.3 var(--serif); }
  .bar { height: 8px; background: var(--wash); border-radius: 4px; margin: 8px 0 2px; overflow: hidden; }
  .bar span { display: block; height: 100%; background: var(--ink); }
  .stamp { display: inline-block; border: 2px solid var(--stamp); color: var(--stamp); font-weight: 700;
           padding: 2px 10px; border-radius: 4px; transform: rotate(-3deg); }
  .panel { border: 1px solid var(--rule); border-radius: 4px; padding: 16px; background: var(--wash); }
  .panel + .panel { margin-top: 16px; }
  .winner { border: 2px solid var(--ink); padding: 14px 18px; margin: 0 0 20px; }
  .winner b { font: 700 24px/1.3 var(--serif); }
  #msg { margin-top: 18px; padding: 10px 14px; border-left: 4px solid var(--rule); background: var(--wash); min-height: 42px; }
  #msg.error { border-color: var(--stamp); }
  #msg.good { border-color: var(--ok); }
  .hidden { display: none; }
</style>
</head>
<body>
<main>
  <div class="top">
    <div>
      <h1>E-Voting dApp</h1>
      <p class="muted">One registered wallet, one vote, recorded on the Sepolia test network.</p>
    </div>
    <button id="connectBtn">Connect MetaMask</button>
  </div>

  <div class="setup">
    <div>
      <label for="addr">Voting contract address</label>
      <input id="addr" placeholder="0x... (copy it from Remix after deploying Voting.sol)" spellcheck="false">
    </div>
    <button id="loadBtn" class="ghost">Load election</button>
  </div>

  <div class="status" id="status">
    <div><small>Your wallet</small><b id="stAccount">Not connected</b></div>
    <div><small>Your role</small><b id="stRole">-</b></div>
    <div><small>Election phase</small><b id="stPhase">-</b></div>
    <div><small>Votes cast</small><b id="stTotal">-</b></div>
  </div>

  <div id="winnerBox" class="winner hidden"></div>

  <div class="grid">
    <section>
      <h2>Ballot</h2>
      <div id="candidates"><p class="muted">Connect your wallet and load the election to see the candidates.</p></div>
    </section>

    <section id="adminSection" class="hidden">
      <h2>Admin panel</h2>
```

```
      <div class="panel" id="panelAdd">
        <label for="candName">Add a candidate</label>
        <input id="candName" placeholder="Candidate name">
        <p style="margin-top:8px"><button id="addCandBtn">Add candidate</button></p>
      </div>
      <div class="panel" id="panelVoters">
        <label for="voterList">Register voters (wallet addresses)</label>
        <textarea id="voterList" rows="4" placeholder="One address per line, or separated by commas"></textarea>
        <p style="margin-top:8px"><button id="regBtn">Register voters</button></p>
      </div>
      <div class="panel">
        <p class="muted" id="phaseHelp"></p>
        <button id="startBtn">Start voting</button>
        <button id="endBtn" class="ghost">End voting</button>
      </div>
    </section>
  </div>

  <div id="msg" role="status" aria-live="polite">Install MetaMask, switch to Sepolia, then connect.</div>
</main>

<script src="https://cdn.jsdelivr.net/npm/ethers@6.13.4/dist/ethers.umd.min.js"></script>
<script>
const SEPOLIA_CHAIN_ID = "0xaa36a7";
const PHASES = ["Registration", "Voting", "Ended"];
const ABI = [
  "function admin() view returns (address)",
  "function phase() view returns (uint8)",
  "function totalVotes() view returns (uint256)",
  "function isRegistered(address) view returns (bool)",
  "function hasVoted(address) view returns (bool)",
  "function getCandidates() view returns (tuple(uint256 id, string name, uint256 voteCount)[])",
  "function getWinner() view returns (string name, uint256 votes)",
  "function addCandidate(string name)",
  "function registerVoters(address[] voters)",
  "function startVoting()",
  "function endVoting()",
  "function vote(uint256 candidateId)"
];

let provider, signer, contract, account;
const $ = (id) => document.getElementById(id);

function say(text, kind) {
  const m = $("msg");
  m.textContent = text;
  m.className = kind || "";
}

function errText(e) {
  return e.reason || (e.info && e.info.error && e.info.error.message) || e.shortMessage || e.message || String(e);
}

function short(a) { return a.slice(0, 6) + "..." + a.slice(-4); }

async function connect() {
  if (!window.ethereum) { say("MetaMask was not found. Install the MetaMask extension and reload this page.", "error"); return; }
  try {
    await window.ethereum.request({ method: "eth_requestAccounts" });
    const chain = await window.ethereum.request({ method: "eth_chainId" });
    if (chain !== SEPOLIA_CHAIN_ID) {
      await window.ethereum.request({ method: "wallet_switchEthereumChain", params: [{ chainId: SEPOLIA_CHAIN_ID }] });
    }
    provider = new ethers.BrowserProvider(window.ethereum);
    signer = await provider.getSigner();
    account = await signer.getAddress();
    $("stAccount").textContent = short(account);
    $("connectBtn").textContent = "Connected";
    say("Wallet connected on Sepolia. Paste the contract address and load the election.", "good");
    if ($("addr").value) await loadContract();
  } catch (e) { say(errText(e), "error"); }
}

async function loadContract() {
  const a = $("addr").value.trim();
  if (!signer) { say("Connect your wallet first.", "error"); return; }
  if (!ethers.isAddress(a)) { say("That is not a valid contract address.", "error"); return; }
  contract = new ethers.Contract(a, ABI, signer);
  try { localStorage.setItem("votingAddress", a); } catch (e) {}
  await refresh();
}

async function refresh() {
  if (!contract) return;
  try {
    const [admin, phase, total, reg, voted, cands] = await Promise.all([
      contract.admin(), contract.phase(), contract.totalVotes(),
      contract.isRegistered(account), contract.hasVoted(account), contract.getCandidates()
    ]);
    const p = Number(phase);
    const isAdmin = admin.toLowerCase() === account.toLowerCase();

    $("stPhase").textContent = PHASES[p];
    $("stTotal").textContent = total.toString();
    $("stRole").textContent = isAdmin ? "Admin" : reg ? (voted ? "Registered voter (already voted)" : "Registered voter") : "Not
registered";

    renderCandidates(cands, p, reg, voted, Number(total));
    renderAdmin(isAdmin, p);
    await renderWinner(p);
```

```
  } catch (e) { say("Could not read the contract. Check the address and that MetaMask is on Sepolia. " + errText(e), "error"); }
}

function renderCandidates(cands, p, reg, voted, total) {
  const box = $("candidates");
  box.textContent = "";
  if (cands.length === 0) {
    const none = document.createElement("p");
    none.className = "muted";
    none.textContent = "No candidates yet. The admin adds them during Registration.";
    box.appendChild(none);
    return;
  }
  cands.forEach((c) => {
    const votes = Number(c.voteCount);
    const pct = total > 0 ? Math.round((votes / total) * 100) : 0;

    const wrap = document.createElement("div"); wrap.className = "candidate";
    const row = document.createElement("div"); row.className = "row";
    const name = document.createElement("span"); name.className = "name"; name.textContent = c.name;
    row.appendChild(name);

    if (p === 1) {
      const btn = document.createElement("button");
      btn.textContent = "Vote";
      btn.disabled = !reg || voted;
      btn.title = !reg ? "Your wallet is not registered" : voted ? "You have already voted" : "";
      btn.addEventListener("click", () => send(() => contract.vote(c.id), "Vote"));
      row.appendChild(btn);
    }
    wrap.appendChild(row);

    const bar = document.createElement("div"); bar.className = "bar";
    const fill = document.createElement("span"); fill.style.width = pct + "%";
    bar.appendChild(fill); wrap.appendChild(bar);

    const count = document.createElement("small"); count.className = "muted";
    count.textContent = votes + (votes === 1 ? " vote" : " votes") + " (" + pct + "%)";
    wrap.appendChild(count);
    box.appendChild(wrap);
  });
  if (voted) {
    const s = document.createElement("p");
    const st = document.createElement("span"); st.className = "stamp"; st.textContent = "VOTE RECORDED";
    s.style.marginTop = "16px"; s.appendChild(st); box.appendChild(s);
  }
}

function renderAdmin(isAdmin, p) {
  $("adminSection").classList.toggle("hidden", !isAdmin);
  if (!isAdmin) return;
  $("panelAdd").classList.toggle("hidden", p !== 0);
  $("panelVoters").classList.toggle("hidden", p !== 0);
  $("startBtn").disabled = p !== 0;
  $("endBtn").disabled = p !== 1;
  $("phaseHelp").textContent =
    p === 0 ? "Add at least 2 candidates and register voters, then start voting." :
    p === 1 ? "Voting is open. End voting to publish the result." : "The election has ended.";
}

async function renderWinner(p) {
  const w = $("winnerBox");
  if (p !== 2) { w.classList.add("hidden"); return; }
  const r = await contract.getWinner();
  w.textContent = "";
  const label = document.createElement("p"); label.className = "muted"; label.textContent = "Result";
  const nm = document.createElement("b"); nm.textContent = r[0];
  const v = document.createElement("p"); v.textContent = r[1].toString() + " votes";
  w.appendChild(label); w.appendChild(nm); w.appendChild(v);
  w.classList.remove("hidden");
}

async function send(makeTx, label) {
  try {
    say(label + ": confirm the transaction in MetaMask...");
    const tx = await makeTx();
    say(label + ": waiting for the network to confirm (about 15 seconds)...");
    await tx.wait();
    say(label + " confirmed. Transaction: " + tx.hash, "good");
    await refresh();
  } catch (e) { say(label + " failed: " + errText(e), "error"); }
}

$("connectBtn").addEventListener("click", connect);
$("loadBtn").addEventListener("click", loadContract);
$("addCandBtn").addEventListener("click", () => {
  const n = $("candName").value.trim();
  if (!n) { say("Enter a candidate name.", "error"); return; }
  send(() => contract.addCandidate(n), "Add candidate").then(() => { $("candName").value = ""; });
});
$("regBtn").addEventListener("click", () => {
  const list = $("voterList").value.split(/[\s,;]+/).filter(Boolean);
  if (list.length === 0) { say("Paste at least one wallet address.", "error"); return; }
  const bad = list.find((x) => !ethers.isAddress(x));
  if (bad) { say("Not a valid address: " + bad, "error"); return; }
  send(() => contract.registerVoters(list), "Register voters").then(() => { $("voterList").value = ""; });
});
$("startBtn").addEventListener("click", () => send(() => contract.startVoting(), "Start voting"));
$("endBtn").addEventListener("click", () => send(() => contract.endVoting(), "End voting"));
if (window.ethereum) {
```

```
  window.ethereum.on("accountsChanged", () => location.reload());
  window.ethereum.on("chainChanged", () => location.reload());
}
try { const saved = localStorage.getItem("votingAddress"); if (saved) $("addr").value = saved; } catch (e) {}
setInterval(refresh, 12000);
</script>
</body>
</html>
```